

Test Doubles em Testes Unitários (Mock, Stub, Fake e Spy)

Em testes unitários, muitas vezes não queremos depender de elementos reais como:

- Banco de dados
- APIs externas
- Serviços complexos

Para isso, utilizamos **Test Doubles**, que são objetos que **simulam comportamentos reais**.

1. Stub (retorno controlado)

Um **Stub** é usado quando queremos **controlar o retorno de um método**. Ele não valida comportamento, apenas responde algo previamente definido.

Exemplo

```
ContaRepository repo = mock(ContaRepository.class);

when(repo.findById(1L))
    .thenReturn(Optional.of(new Conta(1L, "João", new BigDecimal("100.00"))));
```

Explicação

```
when(repo.findById(1L))
```

➡ Quando o método for chamado...

```
.thenReturn(...)
```

➡ ...retorne esse valor específico

✓ Quando usar?

- Quando você precisa de **dados controlados**
- Quando quer **isolar o teste do banco de dados**

2. Mock (verificação de comportamento)

Um **Mock** é usado para **verificar se determinados métodos foram chamados**. Aqui o foco NÃO é o retorno, mas sim a interação.

Exemplo

```
verify(repo, times(1)).save(conta);
```

Explicação

```
verify(repo, times(1))
```

➡ Verifica quantas vezes o método foi chamado

```
.save(conta)
```

➡ Verifica se foi chamado com esse parâmetro

✓ Quando usar?

- Quando você quer garantir que uma ação ocorreu
- Muito usado em **regras de negócio**

3. Fake (implementação simplificada)

Um **Fake** é uma implementação **real**, porém simplificada. Ele possui lógica de funcionamento, mas não é o sistema real.

Exemplo

```
class ContaRepositoryFake implements ContaRepository {  
  
    private Map<Long, Conta> banco = new HashMap<>();  
  
    @Override  
    public Optional<Conta> findById(Long id) {  
        return Optional.ofNullable(banco.get(id));  
    }  
  
    @Override
```

```
    public Conta save(Conta conta) {  
        banco.put(conta.getId(), conta);  
        return conta;  
    }  
}
```

Explicação

- Usa um **Map** como banco de dados
- Funciona de verdade, mas sem banco real

✓ Quando usar?

- Quando precisa testar fluxo completo
- Quando quer algo mais próximo do real

4. Spy (objeto parcial)

Um **Spy** é um objeto real que pode ser **monitorado ou alterado parcialmente**. Ele executa o comportamento real, mas permite ver interações.

Exemplo

```
List<String> lista = new ArrayList<>();  
List<String> spyLista = spy(lista);  
  
spyLista.add("A");  
  
verify(spyLista).add("A");
```

Explicação

- O método **add** realmente executa
- Mas também conseguimos verificar a chamada

✓ Quando usar?

- Quando quer usar comportamento real
- Mas ainda assim verificar interações

Exemplo Completo (Mock + Stub)

```
@Test
void deveChamarSaveAoTransferir() {
    ContaRepository repo = mock(ContaRepository.class);

    Conta origem = new Conta(1L, "João", new BigDecimal("100.00"));
    Conta destino = new Conta(2L, "Maria", new BigDecimal("50.00"));

    // ♦ STUB (define comportamento)
    when(repo.findById(1L)).thenReturn(Optional.of(origem));
    when(repo.findById(2L)).thenReturn(Optional.of(destino));

    ContaService service = new ContaService(repo);
    service.transferir(1L, 2L, new BigDecimal("50.00"));

    // ♦ MOCK (verifica comportamento)
    verify(repo, times(1)).save(origem);
    verify(repo, times(1)).save(destino);
}
```

Análise do exemplo

♦ Stub

```
when(repo.findById(...))
```

➡ Define o retorno esperado

♦ Mock

```
verify(repo).save(...)
```

➡ Verifica se o método foi chamado

Resumo Geral

Tipo	Objetivo	Lógica real	Verifica interação
Stub	Retornar dados	✗ Não	✗ Não

Mock	Validar chamadas	✗ Não	✓ Sim
Fake	Simular sistema real	✓ Sim	✗ Não
Spy	Parcial (real + controle)	✓ Sim	✓ Sim

Boas práticas

- ✓ Use **Stub** para controlar dados
- ✓ Use **Mock** para validar comportamento
- ✓ Evite misturar muitas responsabilidades no mesmo teste
- ✓ Prefira testes simples e objetivos

O uso correto de **Mock**, **Stub**, **Fake** e **Spy** permite:

- Testes mais rápidos
- Menor dependência de infraestrutura
- Maior confiabilidade no código

Esses conceitos são fundamentais para escrever testes de qualidade e são muito utilizados no mercado.